

Practical and Efficient x86-64 Emulation on RISC-V

Xiongchuan Tan*
txc24@mails.tsinghua.edu.cn
RIOS Lab, Tsinghua University
Shenzhen, China

Yang Liu*
liuyang22@iscas.ac.cn
Institute of Software,
Chinese Academy of Sciences
Beijing, China

Sebastien Chevalier
sebastien.chev@gmail.com
Unaffiliated
Villefontaine, France

Yangyu Chen
cyy@cyyself.name
College of Computer Science,
Chongqing University
Chongqing, China

Xiaoyi Liu
xiaoyi-l23@mails.tsinghua.edu.cn
Department of Computer Science,
Tsinghua University
Beijing, China

Haohuan Fu
haohuan@tsinghua.edu.cn
RIOS Lab, Tsinghua University
Shenzhen, China

Abstract

As RISC-V increasingly gains traction across various computing domains, the need to run x86-64 applications on RISC-V platforms becomes critical.

Binary translation plays a pivotal role in ensuring software portability across diverse Instruction Set Architectures (ISAs). However, the architectural differences between the widely adopted x86-64 and the emerging RISC-V platforms challenge effective translation while maintaining performance and compatibility. To address this challenge, we extend Box64 to enable efficient execution of x86-64 software on RISC-V systems. Our approach leverages a suite of both general and RISC-V-specific optimization techniques—including an innovative Total Store Ordering (TSO) emulation method, refined indirect and conditional branch optimizations, and dynamic library acceleration—to drastically reduce emulation overhead, improve execution efficiency, and maintain extensive application compatibility. Evaluations show that our solution only incurs an average performance penalty of less than 40% compared to native execution, outperforms QEMU-TCG by 3.3×, and robustly supports a wide range of applications from office suites to video games.

CCS Concepts: • Software and its engineering → Just-in-time compilers; Virtual machines; • Computer systems organization → Reduced instruction set computing; Complex instruction set computing; Processors and memory architectures.

Keywords: Dynamic Binary Translation, RISC-V, x86-64, Memory Consistency Models

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License.

EUROSYS '26, Edinburgh, Scotland UK

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2212-7/26/04

<https://doi.org/10.1145/3767295.3803574>

ACM Reference Format:

Xiongchuan Tan, Yang Liu, Sebastien Chevalier, Yangyu Chen, Xiaoyi Liu, and Haohuan Fu. 2026. Practical and Efficient x86-64 Emulation on RISC-V. In *European Conference on Computer Systems (EUROSYS '26)*, April 27–30, 2026, Edinburgh, Scotland Uk. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3767295.3803574>

1 Introduction

The rapid ascent of RISC-V as an open-standard Instruction Set Architecture (ISA) is reshaping the hardware landscape, from edge computing to High Performance Computing (HPC) domains. However, its most significant barrier to widespread adoption remains the “software gap”: a lack of native support for the vast ecosystem of applications built for mature architectures like x86. While binary translation offers a promising path to bridge this gap, existing general-purpose solutions like QEMU [6] often incur prohibitive performance overhead or lack compatibility with complex, real-world software, rendering them unsuitable for practical daily use. This creates an urgent need for a high-performance, high-compatibility binary translation system tailored for the RISC-V ecosystem.

The primary challenge in building such a system lies in the significant architectural chasm between x86 and RISC-V, which represents a fundamental paradigm shift in ISA design. As illustrated in Figure 1, this disparity is more pronounced than that between x86 and Arm. Esoteric x86 features, such as the 80-bit extended precision of x87 floating-point and unaligned atomic operations (shown in red), are notoriously difficult to emulate efficiently on RISC-V. Furthermore, significant variations in features like sub-word atomics and Single Instruction Multiple Data (SIMD) capabilities (shown in yellow) exist not only between x86 and RISC-V but also across the fragmented landscape of RISC-V cores with different extension sets. These architectural mismatches demand novel and specialized translation strategies to achieve both correctness and performance.

A second critical challenge arises from the disparity in memory consistency models, which is crucial for the correct

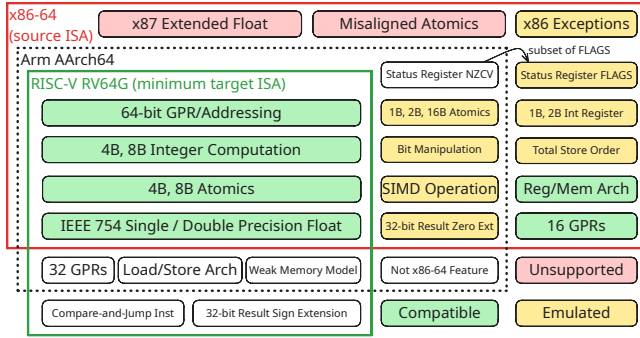


Figure 1. ISA feature comparison and the coverage of our work. AArch64 is included here to act as a reference.

execution of multithreaded applications. The x86 architecture employs a strong Total Store Ordering (TSO) model, whereas RISC-V, like Arm, uses a Weak Memory Model (WMM). To guarantee correctness, a translator must insert explicit memory barrier instructions into the generated code, which can severely degrade performance. Therefore, a key research question is how to emulate TSO semantics with minimal overhead, employing memory barriers judiciously only when strictly necessary, rather than conservatively enforcing them.

To address these challenges, we extend Box64 [31], our original x86-to-Arm binary translator, to build an efficient and robust bridge between x86 and RISC-V ecosystems. By focusing on application-level emulation, our system bypasses the complexities of full hardware or kernel virtualization, allowing us to concentrate on advanced translation and optimization techniques. The efficacy of our approach is demonstrated by its remarkable compatibility with a vast array of x86-64 applications.

The contributions of this paper are as follows:

- We extend **Box64** to build the first Dynamic Binary Translation (DBT) system for running *real-world, complex, and unmodified* x86-64 applications (e.g., WPS Office, desktop games) on RISC-V.
- We develop a **high-performance translation engine** that achieves up to 3.3× the performance of QEMU and 67% of native execution. Its co-design of a JIT compiler and a direct API wrapping mechanism provides both high throughput and direct GPU compatibility.
- We design a **unified translation framework** that bridges the architectural gap between x86-64 and the fragmented RISC-V hardware landscape, providing robust software emulation for critical ISA features (e.g., atomics, memory model, SIMD) to ensure broad compatibility.

- We introduce a **pragmatic and efficient TSO emulation method**. Based on real-world application behavior, it uses selective memory barriers to ensure multi-threaded correctness with significantly lower overhead than state-of-the-art techniques.

2 Background & Motivation

2.1 Binary Translation

Binary translation can be broadly categorized as static or dynamic, depending on whether the source binary is translated ahead-of-time or during program execution. While Static Binary Translation (SBT) benefits from a generous resource budget for Ahead-of-Time (AOT) compilation, accurately resolving the complete control flow of a program without runtime information is notoriously challenging [36]. This difficulty is exacerbated by complexities of the x86 ISA, such as its variable-length instructions, the prevalence of Self-Modifying Code (SMC)¹, and the ambiguous mixture of code and data.

In contrast, DBT operates as a Just-in-Time (JIT) compiler that translates guest instructions on-the-fly. It works in a discovery-translation-execution loop, utilizing a translation cache to reuse previous translation results. Because the compilation stage occurs during the execution of the emulated program, it faces stringent real-time performance constraints not present in SBT. There have been attempts to enhance JIT compilation, such as incremental trace building [8] for frequently executed paths, and persistent translation caches [22] that mimic SBT behavior.

2.2 Memory Consistency Model

Table 1. Reordering Behavior Summary [24, 35]

	x86-TSO	RVWMO
Load $\prec_p$ Load		Y
Load $\prec_p$ Store		Y
Store $\prec_p$ Store		Y
Store $\prec_p$ Load	Y	Y

$A \prec_p B$: A is earlier than B in *program* order.

$A \prec_m B$: A is observed before B in *memory* order.

When reordering is allowed (Y), $A \prec_p B \not\Rightarrow A \prec_m B$.

The memory consistency model defines the rules for how concurrent operations on shared memory are observed by different threads. In multithreaded programs, CPUs can cause the execution order of instructions to differ from the program order, affecting the perceived order of memory events across threads [24]. As summarized in Table 1, x86-TSO only allows stores to be reordered after loads, whereas WMM architectures allow the CPU to enhance parallelism by reordering

¹We use SMC as a broad term to denote the modification and generation of code during runtime, including JIT.

a wider variety of memory accesses. Directly translating memory operations without fences from x86-TSO to WMM architectures like RISC-V can lead to data races or crashes in multithreaded applications, as the original memory ordering guarantees are lost. Consequently, because most WMM architectures lack specialized hardware for TSO emulation (such as that found in Apple Silicon [2]), software layers like the compiler or binary translator must insert explicit memory barriers. This preserves correct memory access ordering and ensures proper synchronization between threads.

2.3 Challenges of x86-on-RISC-V Emulation

For x86-to-RISC-V DBT systems, significant challenges stem from emulating x86-specific ISA features that lack a direct equivalent in RISC-V, such as floating-point processing, status registers, complex atomic operations, and TSO emulation. The *reduced* nature of RISC-V and its *fragmented* hardware landscape further exacerbate these challenges.

Floating-Point Unit (FPU) emulation is a classical bottleneck in emulators. In the x86 ISA, an additional hardware floating-point (FP) type with extended 80-bit precision is provided alongside canonical 32-bit and 64-bit IEEE FP types. The legacy x87 instruction subset is capable of performing calculations with these precisions, which are dynamically selected by a status register at runtime. Many 32-bit x86 applications rely on specific x87 behaviors, complicating the emulation design. Furthermore, mismatches in NaN generation and rounding behaviors between platforms represent another major corner case, requiring additional guard code to ensure consistent execution.

Efficient emulation of the x86 FLAGS register requires careful translator optimization. Unlike Arm and LoongArch, which feature hardware extensions to assist with x86 status flag generation, current RISC-V implementations must rely entirely on software emulation. Furthermore, limitations of bit-manipulation instructions in the standard RISC-V ISA make efficient FLAGS emulation highly challenging.

Regarding atomic memory operations, while sub-word atomics can be implemented using larger atomic sizes, x86 does not require memory alignment for instructions like `LOCK CMPXCHG`—a strict requirement in the standard RISC-V A extension. Similarly, the 128-bit double-word atomic operation `CMPXCHG16B` has no direct equivalent in the current RISC-V ISA extensions available on our target hardware, and a multi-instruction software implementation suffers from potential tearing. Without hardware assistance, guaranteeing rigorous software atomicity incurs significant overhead.

Some weakly-ordered platforms provide a hardware TSO mode (e.g., Apple’s M-series chips) or special data transfer instructions to mimic TSO memory accesses (e.g., Arm’s Release Consistency processor consistent (RCpc) instructions). Without such hardware assistance, TSO emulation must rely entirely on memory fence instructions. Consequently, atomic

operations and TSO emulation demand significant design tradeoffs between performance and correctness.

We have chosen RV64G as the minimal ISA baseline to cover broad hardware compatibility. Support for additional instruction extensions can be optionally enabled to leverage available host capabilities. This complicates the code generation design, particularly for FP/SIMD instructions. While x86 instructions use a single set of XMM registers, our design requires synchronization across three locations: memory (the emulation state), scalar FP registers (for scalar emulation), and vector registers (for optional vector emulation).

3 Related Works

3.1 x86 Emulation on Other Platforms

Compared to the nascent ecosystem on RISC-V, mature DBT systems for x86 emulation already exist on other platforms. To bridge the critical gap in x86-on-RISC-V emulation, we adapt and advance successful design patterns from these state-of-the-art translators, pioneering their application within the RISC-V context.

The canonical example of a many-to-many DBT system is QEMU [6], which prioritizes functional correctness—such as meticulously emulating exact FPU behavior—over performance. Its use of a general-purpose Intermediate Representation (IR), the Tiny Code Generator (TCG), makes it highly portable but introduces significant overhead. Consequently, QEMU’s performance is often insufficient for demanding emulation tasks, spurring a large body of research focused on performance optimization. These efforts include enhancing its code generation backend [14, 17, 38], integrating it with compiler infrastructures like LLVM [9, 18, 20], minimizing memory model emulation overhead [15, 16], and accelerating dynamic library calls [33, 34]. A notable example of specialization is the Loongson Architecture Translator for x86 (LATX) [28], which heavily tailors QEMU specifically for the x86-to-LoongArch translation task.

Beyond enhancing QEMU, the pursuit of practical performance has led to highly specialized translators, particularly for the mature x86-to-Arm ecosystem. This space includes industry-leading proprietary systems like Apple’s Rosetta 2 [1], Microsoft’s Prism [25], and Huawei’s ExaGear [19], as well as prominent open-source projects such as Box64 [31] and FEX [12].

Key techniques employed by many of these modern translators include static register mapping, unused flag optimization, equivalent instruction mapping, and dynamic library acceleration. Because modern RISC architectures typically feature 32 general-purpose registers—double the 16 available in x86 (excluding APX)—static register mapping schemes are employed to simplify x86 state restoration from host registers and align the Application Binary Interface (ABI) between platforms. Additionally, while many x86 instructions generate status bits in the FLAGS register, most of these bits are

rarely consumed by subsequent instructions. Consequently, modern emulators typically analyze and eliminate redundant flag calculations during translation. Furthermore, by identifying semantic similarities between x86 and the target ISA, many complex x86 instructions (e.g., popcount, SIMD operations) can be directly mapped to native instructions without costly software simulation. Finally, dynamic library acceleration is a distinguishing feature among open-source emulators: while standard QEMU only implements system call emulation for user-mode emulation, Box64 and FEX implement forwarding mechanisms to bridge x86 callers and native callee library functions. This is essential for emulating graphical libraries (e.g., OpenGL and Vulkan), as it allows the emulator to circumvent the intricacies of underlying system call interfaces, which vary significantly across hardware vendors.

3.2 TSO Emulation

To bridge the semantic gap with x86’s native TSO memory model, numerous software and hardware solutions have been developed for emulating TSO on WMM architectures. Generally, increased assistance from the hardware memory subsystem can significantly reduce software emulation complexity.

QEMU implements a highly conservative software approach. When the host memory model is weaker than the emulated architecture and the program executes in a multithreaded environment, memory fence instructions are indiscriminately inserted for every memory access instruction. Subsequent research, such as Risotto [16], identified QEMU’s suboptimal fence selection (which enforces stricter ordering than TSO requires), redundant fence placement, and flaws in read-modify-write emulation that inadvertently violate TSO. To address these issues, Risotto proposed an improved instruction mapping scheme. Building upon Risotto, CrossMapping [15] implements inter-instruction analysis, utilizing a deterministic finite automaton (DFA) to deduce the optimal fence type between different sequences of memory accesses (reads, writes, read-modify-writes, and fences). By rigorously satisfying TSO requirements while minimizing fence insertions, CrossMapping achieves the highest performance for strict software-based TSO emulation among the literature we examined.

Unlike DBT systems, compilers possess *a priori* information regarding synchronization patterns. AtoMig [5] detects implicit synchronization patterns during compilation by examining intermediate LLVM IR, then inserts necessary atomic memory accesses and barriers to address potential memory order issues. In the commercial sphere, Microsoft’s MSVC compiler generates metadata for “volatile” memory accesses to inform the DBT translator of specific code regions where strict TSO semantics are required [27]. This feature has been enabled for x64 code generation by default since

Visual Studio 2019 16.10 and is prevalent in many modern Windows applications.

Leveraging its vertically integrated ecosystem, Apple adopts a hardware-assisted TSO approach. Apple Silicon M-series chips feature a hardware TSO mode that can be dynamically toggled via a specialized control register. In this mode, the processor’s memory model directly complies with TSO by disallowing the additional reordering types permitted by the WMM, thus eliminating the overhead of software emulation entirely. Similarly, the Armv8.3-A architecture introduces the RCpc model [3], which is highly beneficial for emulating x86 TSO semantics. Specifically, Arm’s LDAPR and STLR instructions adhere to TSO’s reordering constraints, unlike standard load and store instructions. FEX leverages these RCpc instructions to replace ordinary memory accesses for efficient TSO emulation. However, these hardware-centric solutions inherently suffer from portability limitations, restricting their applicability to specific platforms—a barrier that software developers cannot overcome on current RISC-V hardware.

4 System Overview

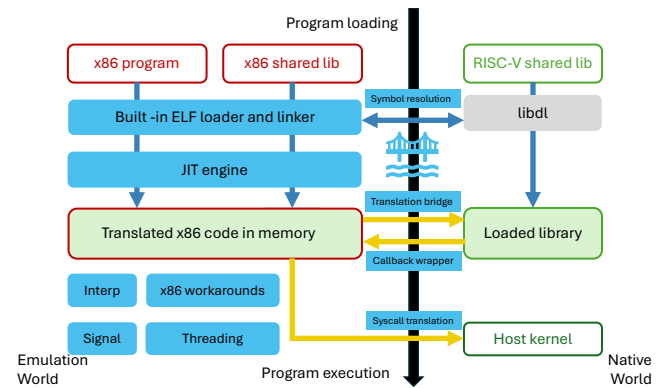


Figure 2. Overview of Box64

Box64 is a DBT system designed for both performance and compatibility. Figure 2 illustrates the overall architecture of our system. The execution process begins with a custom internal linker that loads the main program image and resolves dependencies for both guest (x86) and host (RISC-V) dynamic libraries. Once execution starts, a central dispatcher intercepts the guest instruction pointer. If the target x86 code block is not found in the translation cache, it is passed to the JIT compiler. The resulting RISC-V code is then cached and linked back to the source x86 address, allowing future executions to bypass the dispatcher and execute the native code directly. The core translator is a lightweight, four-pass JIT compiler that operates without a formal IR to minimize translation overhead. It decodes x86 instructions linearly while performing several on-the-fly optimizations, such as FLAGS calculation reduction, direct linking of internal jump

targets, and FP/SIMD register allocation. While this streamlined design minimizes translation overhead and generates efficient code, supporting complex real-world applications requires several additional components to handle system-level intricacies.

A cornerstone of our system’s design is its high-level emulation strategy, which prioritizes library-level interoperability over low-level system call translation. This is enabled by our custom dynamic linker, which handles cross-architecture dependency resolution—a task QEMU offloads to an external guest linker. During this process, lightweight “glue code” is generated to manage the transition between guest and host calling conventions. This design allows us to intercept library calls from the guest application and redirect them to native host libraries. This approach yields two major benefits: it circumvents redundant translation of common library functions, and it avoids the complexity of emulating low-level system calls, particularly the notoriously volatile device ioctls. Consequently, our system call translation layer is significantly thinner than QEMU’s. Critical tasks like GPU driver communication are handled directly by the host’s native libraries without emulator intervention, leading to substantially better graphics performance and compatibility. This interoperability is managed by translating arguments during each cross-architecture call, guided by a function signature database.

Of course, a practical DBT system requires principled design trade-offs. For instance, to ensure accurate signal handling, we maintain a sequential mapping between guest and host instructions, foregoing some reordering optimizations. Similarly, based on the observation that most target applications do not require full 80-bit precision, our x87 emulation uses 32-bit or 64-bit hardware floating-point instructions, avoiding the severe performance penalty of soft-float emulation. This philosophy of pragmatic emulation extends to our memory model. Rather than pursuing a rigorously complete but potentially slow emulation of all corner-case memory behaviors, we focus on implementing a model that is correct and performant for the vast majority of real-world, multi-threaded software. This approach, detailed in Section 6.3 and Section 7, allows us to achieve robust compatibility with high performance.

5 High-Performance JIT Engine Design

5.1 Branch Optimization

While direct branches can often be inlined or resolved with minimal overhead in our DBT, indirect branches present a significant performance bottleneck. Each indirect jump requires a costly lookup in a data structure that maps guest (x86) code addresses to their translated host (RISC-V) code addresses. Our system employs a multi-level jump table, similar in structure to a page table, which represents a design trade-off between the memory footprint, and the lookup

latency. Other systems may use other data structures with a larger memory footprint, such as hash tables, or even a flat table like Tiaozhuan [21].

The most frequent and therefore most critical indirect branch to optimize is the function return instruction (RET). Rather than performing a costly table lookup for every return, our approach aims to leverage the host CPU’s specialized hardware for branch prediction, specifically the Return Address Stack (RAS). The core idea, first introduced in systems like MAMBO-X64 [10], is to translate the guest CALL/RET instruction pair into the host’s native counterparts, JAL/JALR in RISC-V. This allows the host hardware to predict return addresses with high accuracy.

To implement this, we maintain a software shadow stack that operates in parallel with the guest’s own stack. When a guest CALL instruction is translated to a host JAL, we push the translated host return address onto our shadow stack. When a guest RET is encountered, we compare the return address found on the guest stack with the address we expect from our shadow stack. If they match—the common case—we continue normal execution by executing a fast native JALR to translated code after the CALL instruction, perfectly utilizing the hardware RAS. However, if a mismatch occurs, which can happen due to exceptions or non-standard control flow like `set jmp/long jmp`, we de-optimize and fall back to the slow path to resynchronize emulator execution with the actual guest program counter. This technique effectively transforms the most common indirect branch from an expensive lookup into a highly predictable native branch, yielding substantial performance gains.

5.2 Efficient FLAGS Emulation

The x86 ISA’s reliance on a centralized FLAGS register for status and control bits creates a significant architectural impedance mismatch with RISC-V, which lacks a direct hardware equivalent. A naive emulation approach—calculating all six status flags (CF, PF, AF, ZF, SF, OF) after every relevant x86 instruction—would be prohibitively expensive due to the software overhead. The foundation of our efficient solution lies in a key observation: the vast majority of FLAGS updates are “dead,” as the generated bits are only consumed by a small subset of subsequent instructions, primarily conditional branches. Our goal, therefore, is to eliminate these redundant computations.

To achieve this, we implement a lazy, on-demand computation strategy for status flags, operating directly on the guest instruction stream to minimize JIT compilation overhead. Within a single basic block, we perform a backward data flow analysis to determine the liveness of each status bit at every instruction. Guided by a specification table (e.g., Table 2) that details how each x86 instruction reads and writes the flags, we generate code to compute only the specific flags that are actually consumed later in the block. At the boundaries

Table 2. Example of FLAGS Specification Table

Instruction	0	1	2	3	4	5
ADD/SUB/CMP	M	M	M	M	M	M
ADC/SBB	M	M	M	M	M	X
MUL	M	U	U	U	U	M
MOV/CALL/RET/JMP						
JZ/SETZ/CMOVZ			T			

Notation: M: Set only. X: Read and set.

T: Read only. Empty: Unaffected.

U: Undefined in specification.

between blocks, this laziness is extended: instead of materializing the full FLAGS register, we save only the necessary intermediate operands of the last flag-setting instruction. The responsibility for calculating the final flag value is deferred to the successor block, which will only perform the calculation if it actually needs the result. This principle is further applied to CALL and RET instructions; based on the common ABI convention that status flags are not preserved across function calls, we generate no FLAGS-related code at these boundaries by default².

Beyond the frequently updated status bits, the FLAGS register also contains control bits that alter processor behavior, notably the Direction Flag (DF) and the Trap Flag (TF). TF presents a great challenge for DBT systems. When TF is set, the processor must trigger a single-step exception after every instruction, a feature primarily used by debuggers³. To implement this correctly without penalizing normal execution, our system monitors all writes to the FLAGS register. If a write sets the TF bit, we de-optimize execution and transfer control to a slower, more precise interpreter. This interpreter executes instructions one by one, manually checking for the single-step exception until the TF is cleared, at which point fast, JIT-compiled execution resumes.

5.3 Register Mapping and Allocation

A key design principle of our JIT is a static, ABI-aware register mapping. The larger register file of RISC-V allows us to dedicate a host register to each of the x86 general-purpose registers, the FLAGS register (rflags), and the instruction pointer (rip), as shown in Table 3. This static, one-to-one mapping provides a crucial advantage for system robustness: it enables the precise and instantaneous reconstruction of the x86 machine state (ucontext⁴) from a RISC-V context. This is essential for correctly handling signals—where a guest signal handler may need to inspect and modify the guest

register state—and for safely aborting translated code blocks mid-execution while preserving a coherent x86 state.

Furthermore, our mapping is not arbitrary; it is carefully designed to align with the System V ABIs of both x86 and RISC-V, a concept influenced by the Arm64EC ABI [25]. We align callee-saved registers from x86 (e.g., rbx, rbp) with callee-saved registers in RISC-V (s0–s11), and map the respective argument-passing registers (rdi–r9, to a0–a5). This ABI-aware alignment minimizes the number of register-to-register moves required at the boundary between emulated code and native library calls. By ensuring argument and callee-saved registers correspond naturally, we significantly reduce the overhead of cross-architecture function invocation, which will be discussed later in Section 6.5.

Implementing this static mapping also requires navigating low-level architectural idiosyncrasies. A prime example is our handling of the Overflow Flag (OF) within the emulated FLAGS register. Our JIT uses the standard RISC-V ORI instruction to efficiently set status bits. However, ORI uses a sign-extended 12-bit immediate, making it impossible to set the 12th bit (bit 11) independently—the exact position of the OF in the x86 specification. To circumvent this, we internally relocate the OF to an unused bit within our host FLAGS register that is compatible with the ORI encoding. When the entire FLAGS register is accessed, such as by PUSHF or POPF instructions, our translator emits code to transparently move the OF bit back to its architecturally-correct position.

Table 3. Register Mapping and Assignment

RISC-V	Box64	RISC-V	Box64
zero	<i>hard-wired zero</i>	a6	rax
ra	<i>RISC-V ra</i>	a7	scratch
sp	<i>RISC-V sp</i>	s2	r12
gp	<i>RISC-V gp</i>	s3	r13
tp	<i>RISC-V tp</i>	s4	r14
t0	<i>x87 control</i>	s5	r15
t1	<i>scratch</i>	s6	rip
t2	<i>scratch</i>	s7	rflags
s0	rbp	s8	rbx
s1	rsp	s9	emu struct ptr
a0	rdi 1st arg	s10	r10
a1	rsi 2nd arg	s11	r11
a2	rdx 3rd arg	t3	scratch
a3	rcx 4th arg	t4	scratch
a4	r8 5th arg	t5	scratch
a5	r9 6th arg	t6	scratch

Notation: (in System V ABI and RISC-V ELF psABI)

argument registers, callee-saved registers

x86 registers, Box64 usage

²DRM-protected software, which often subverts standard conventions, is a notable exception. For such cases, our system can be configured to perform a full FLAGS calculation across function calls.

³Also DRM software for anti-debugging.

⁴A machine-specific representation of the saved register state.

5.4 Handling Conditional Operations on RISC-V

A frequently occurring pattern in the x86 ISA is the pairing of two instructions: a FLAGS-defining instruction (such as CMP or TEST) followed by a FLAGS-consuming conditional operation (like a conditional jump, Jcc, or conditional move, CMOVcc). A conservative and naive one-to-one translation of this pattern is costly on RISC-V. Due to the absence of a hardware FLAGS register, the first instruction would require a lengthy sequence of software instructions to compute and store the status flags, and the second would require another sequence to load and test them, potentially amounting to dozens of instructions for a simple comparison.

Our JIT engine tackles this challenge with a two-level optimization strategy. The first level is our **lazy flag evaluation**, as described in the previous section. This baseline optimization ensures that we only ever compute status bits when they are known to be live, significantly reducing the overhead compared to a naive approach. Even with this optimization, however, materializing the necessary flags for a conditional operation can still require between 3 to 15 RISC-V instructions, leaving substantial room for improvement.

The second and more powerful level of optimization is **macro-op fusion**. The key insight behind this technique is that the intermediate FLAGS value is often just a transient artifact; the true semantic intent of a CMP rdi, rsi followed by a JAE target is simply “jump to target if rdi is above or equal to rsi.” Our JIT’s data flow analysis detects these adjacent define-use pairs. Instead of translating them separately, it fuses them into a single logical operation. It inspects the operands of the FLAGS-defining instruction and directly generates the corresponding native RISC-V conditional branch (e.g., BGEU rdi, rsi, target), completely bypassing the calculation and storage of any FLAGS bits. As illustrated in Figure 3, this fusion technique is highly effective for common pairs involving Jcc and CMOVcc instructions, reducing a potentially long instruction sequence to as few as one or two native instructions, yielding major gains in both performance and code density.

6 Bridging Architectural Gaps for Compatibility

6.1 A Unified Approach for RISC-V Extensions

The modular architecture of the RISC-V ISA [35], while flexible, presents a significant challenge for binary translation: a fragmented hardware landscape. A robust translator must guarantee compatibility on minimally-configured cores while also opportunistically exploiting powerful optional extensions for performance. To address this, our system employs a unified, two-level code generation strategy that ensures both broad compatibility and high performance.

The first level of our strategy establishes a **universal baseline** to guarantee that our translator can run on any common application-class RISC-V processor. We define this

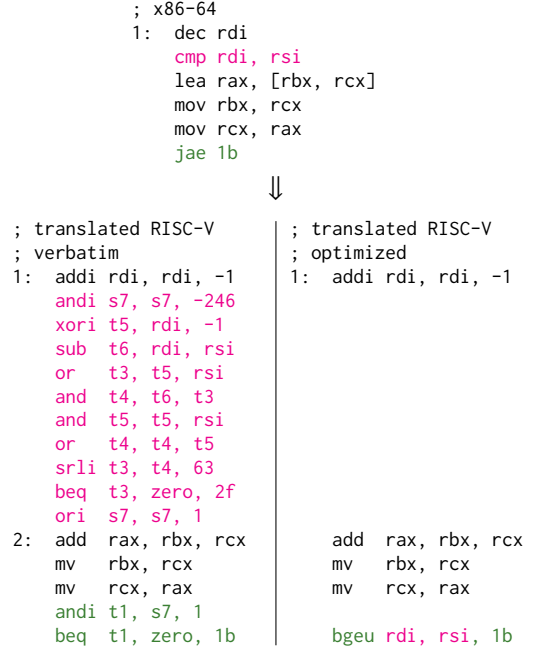


Figure 3. Example of Macro-Op Fusion

baseline as the RV64G instruction set (see Table 4), which is powerful enough to emulate the entire x86-64 instruction set. Our JIT can operate exclusively within this subset, ensuring maximum portability. Notably, we deliberately exclude the optional C (Compressed) extension from our code generation, as its restrictive instruction encoding and limited register access greatly complicates JIT compiler design, and its benefits can be compensated by other complex extensions.

Table 4. Related RISC-V Extensions

Name	Description
RV64I	Base integer instruction set, 64-bit
M	Integer multiplication and division
A	Atomic instructions
F	Single-precision FP operations
D	Double-precision FP operations
Zicsr	Control and status register instructions
Zifencei	Instruction-fetch fence
RV64G	RV64I+M+A+F+D+Zicsr+Zifencei
Zba	Address generation instructions
Zbb	Basic bit-manipulation
Zbs	Single-bit manipulation
V	Vector operations (RVV)
XThread*	Various XuanTie vendor extensions [37]

The second level is **opportunistic acceleration** through dynamic feature detection. At startup, our system queries the host CPU to identify all available standard and non-standard

ISA extensions. The JIT then dynamically enables specialized code generation paths to leverage these features. This allows us to significantly improve performance and code density in several key ways:

- **Increased Code Density:** Fused operations, such as the shift-and-add instructions found in Zba and XuanTie extensions, are used to generate more compact code for common operations like address calculation.
- **Reduced Branching Overhead:** Intrinsic conditional move instructions are favored over multi-instruction branch sequences, which is particularly effective for software FLAGS calculation without polluting the hardware branch predictor.
- **Direct Instruction Mapping:** Specialized x86 instructions for tasks like bit manipulation (e.g. POPCNT) are mapped directly to their efficient single-instruction equivalents in optional RISC-V extensions (e.g., the Zbb and Zbs extension).
- **Efficient SIMD Emulation:** When the Vector (V) extension is detected, x86 SIMD operations are mapped to the powerful RVV instruction set to preserve data-parallel efficiency.

Supporting this wide range of extensions introduces implementation complexities, particularly for stateful extensions like RVV. While stateless extensions can be integrated directly into the code generator, RVV introduces variable-length vector registers and a runtime-configurable element width. This necessitates injecting configuration instructions (VSETVLI) before vector operations, an overhead we mitigate by caching the vector configuration state. A further challenge is the disjoint nature of the floating-point and vector register files in RISC-V, unlike the unified registers in modern x86. Our solution maps scalar x86 FP operations to the F/D extensions and SIMD operations to RVV, inserting explicit register-transfer instructions at the boundary when an emulated x86 SIMD register is used for both types of operations.

6.2 Pragmatic Emulation of the x87 FPU

Our x87 FPU emulation strategy is optimized for performance in legacy 32-bit applications by leveraging the native RISC-V 64-bit double-precision floating-point extension. This approach intentionally avoids a costly, full-precision emulation of the 80-bit extended-precision format, accepting a minor loss of precision that is inconsequential for the vast majority of target applications.

However, this design presents a compatibility issue with a specific software idiom where a FILD (load integer) instruction is followed by a FISTP (store integer and pop) to execute a fast 64-bit integer memory copy. This technique relies on the 64-bit significand of the 80-bit format; direct emulation using 64-bit doubles, which have a smaller significand, results in data truncation. To ensure correctness, our system

statically detects this FILD-FISTP instruction sequence with no intervening arithmetic operations in between, and translates it into equivalent integer load/store operations, thus bypassing the floating-point pipeline and preserving data integrity.

To further enhance performance, we eliminated the runtime overhead of x87 stack simulation. The stack state is tracked statically at compile time, and state transformations are applied at control flow boundaries. While this static method significantly improves performance and code density, it may introduce inaccuracies in rare corner cases.

6.3 Software Emulation for Memory Operations

6.3.1 Emulating x86 Atomic Primitives. The x86 ISA provides a rich set of atomic primitives, supporting operations from 1 to 16 bytes and permitting unaligned access. In contrast, the standard RISC-V A extension is more restrictive, offering only aligned 4-byte and 8-byte atomic operations. To bridge this gap for application compatibility, we employ several pragmatic software emulation techniques. Sub-word (1- and 2-byte) atomics are implemented using wider native RISC-V primitives on an aligned memory word. For the 128-bit CMPXCHG16B instruction, which has no hardware equivalent, we use a global spinlock to serialize access, protecting the 16-byte comparison and swap sequence.

These workarounds represent a trade-off between compatibility and theoretical atomicity. For instance, the spinlock-based CMPXCHG16B is atomic with respect to other spinlock-protected operations but not with native RISC-V atomics. Similarly, for the rare case of unaligned atomic operations, we perform an load-reserved/store-conditional (LR/SC) sequence on an aligned address that overlaps the unaligned access. While this approach is functional for our workloads, it does not provide a true atomicity guarantee against concurrent accesses due to RISC-V's prohibition of intervening memory operations within an LR/SC sequence.

6.3.2 Handling Unaligned Access to Device Memory.

A more common and critical challenge is handling unaligned memory access, particularly for memory-mapped I/O (MMIO) regions like device memory. Unlike x86, RISC-V platforms do not guarantee hardware support for unaligned access to such regions. We discovered this limitation in practice when running DXVK [11], a Direct3D-to-Vulkan translation layer, under our system. DXVK's binaries triggered unexpected bus errors when performing unaligned store operations to a GPU device memory region.

To resolve this issue transparently, we developed a two-stage, signal-and-recompile mechanism.

- **Stage 1: Reactive Fault Handling.** The first time an unaligned write access to a mapped memory region occurs, our custom signal handler is invoked. The handler identifies the faulting instruction, emulates its behavior by breaking the unaligned store into a

sequence of single-byte stores, and then bypasses the original instruction. Before returning, it flags the faulting guest address and invalidates the corresponding JIT translation cache block.

- **Stage 2: Proactive Recompilation.** When the guest program later re-executes this code block, the JIT finds the cache entry invalidated. Triggered by the flag on the guest address, the JIT now re-translates the offending x86 instruction into a safe, decomposed sequence of one-byte stores directly, preventing future faults at this location.

This dynamic approach robustly handles hardware limitations without imposing a global performance penalty, applying the slower, decomposed access pattern only where it is proven necessary.

6.4 Low-Overhead Self-Modifying Code Handling

The x86 architecture implements coherent instruction and data caches for easier programming, while later architectures like RISC-V prefer explicit cache management for better efficiency. For the latter, cache flush instructions are needed to ensure correct behavior when executing JIT-generated code. However, this behavior poses difficulties for software emulation systems. DBT assumes immutable source instruction sequences before caching translated instructions, thus requiring monitoring of all guest memory writes to handle the SMC behavior of x86, which is not feasible for performance reasons.

Our implementation adopts a more efficient strategy to leverage the memory protection of virtual memory: Guest code pages are set to read-only just before being translated. Any write to code pages will trigger a trap signal, allowing the translator to mark all code blocks within the corresponding code page as dirty in the signal handler, set the code page to writable, and then perform the write operation again. By marking code blocks as dirty rather than simply invalidating them, we avoid unnecessary recompilation of other unmodified instruction sequences inside the same page. Whenever a previously translated but dirty guest instruction sequence is executed again, its checksum is compared to the expected value found in the metadata associated with each guest instruction sequence.

In the extreme scenario where the executing guest instruction sequence is altering itself, the translating-and-executing approach will fail. Consequently, the execution of the translated instruction sequence is halted, and our implementation resorts to using an x86 interpreter to fetch and execute actual x86 instructions individually until a branch instruction.

6.5 Dynamic Library Acceleration

Binary translation faces challenges in generating compiled native instructions that match the quality achieved by high-level compilers. Fortunately, the open-source nature of the

Linux ecosystem means that common libraries are widely used across different platforms and are available in native form on the host system, such as the GNU C Library, POSIX Threads, and Simple DirectMedia Layer. This allows for library function calls from the guest program to be forwarded to the host library implementations, ensuring optimized native performance and eliminating the need for complete guest libraries. Additionally, the C ABI remains stable within a single architecture, making inter-platform ABI translations possible.

When a program undergoes dynamic compilation, it experiences two linking processes before execution. The second of these is facilitated by dynamic linkers⁵. Additionally, the program can load dynamic libraries using `libdl` during runtime. DBT systems can exploit the dynamic linking process to identify and replace calls to the original library functions. Our approach mainly uses the population of Procedure Linkage Table (PLT) to accomplish this. However, to connect the guest world with the host world, ABI conversion is required before and after invoking the host library calls.

Parameter translation is an important aspect of ABI conversion. Specifically, we must manage (1) parameters within the integer registers, (2) parameters within the FP/SIMD registers with respect to the difference between x86 and RISC-V, (3) parameters on the stack, (4) variadic parameters, and (5) callback functions. Since symbol names lack type information for global variables, parameters, and return values, a simple internal Interface Description Language (IDL) like Figure 4 is employed to encode this information, which can be automatically generated from C header files. For callback functions reserved for future invocation, our implementation assigns distinct wrappers to each guest function to pretend as native function pointers. Outside of this context, argument translation is a stateless conversion from x86 to RISC-V equivalents. In the case of variadic parameters employed by `printf` families, we examine the format string to retrieve the relevant arguments from their respective positions. The translation of parameters within registers can additionally benefit from the design of register mapping (see Section 5.3).

Nevertheless, directly forwarding dynamic libraries is not always possible for several reasons. For instance, if the guest program is statically compiled, it hinders the intervention of the dynamic linker. Moreover, the guest program might ship its own altered dynamic libraries with specific behavior, or the relevant libraries might lack native equivalents due to being proprietary, such as drivers, or having a relatively unstable ABI, like C++. Emulating library functions related to program execution, such as `clone` and `execvp`, requires special handling, as they might try to launch emulated programs directly, escaping the DBT emulation framework (and

⁵In typical Linux systems based on x86-64 architecture using `glibc`, the dynamic linker is `ld-linux-x86-64.so.2`. Despite its name, which might suggest that it is a dynamic library file, it is actually a statically linked, position-independent executable.

```

// data symbol
// sizeof(_IO_2_1_stdin_) == 224
DATA(_IO_2_1_stdin_, 224)
// text symbol
// void *malloc(size_t)
GO(malloc, pFL)
// text symbol with special handling
// int printf(const char *, ...)
GOM(sprintf, iFEpV)
// text symbol, and weak symbol
// double sin(double)
GOW(sin, dFd)

```

Figure 4. Example of Library Description IDL

would evidently fail to execute since the host CPU is unable to interpret guest instructions). Additionally, the `set jmp` and `long jmp` functions create abnormal control flows that need special attention.

7 Efficient TSO Emulation on a Weakly-Ordered Architecture

Existing software-only TSO emulation has advanced along two largely separate axes: source-level detection of synchronization patterns and instruction-level barrier tuning (see Section 2.2). In practice, these approaches are hard to combine for closed-source binaries, and source-level techniques are inapplicable to most commercial software. Our experiments further show that many complex multithreaded applications run correctly without explicit TSO emulation since libraries like POSIX Threads are from host system, and LOCK prefixed x86 opcodes have an implicit memory barrier, making extra fences useless. When available, compiler-provided annotations (e.g., MSVC “volatile” metadata) can be leveraged to target fences precisely. Motivated by these observations, we design a portable DBT-native framework that integrates synchronization awareness with lightweight, tunable fence insertion for weakly ordered hosts such as RISC-V, and a built-in compatibility profile for real-world applications to balance correctness and performance.

Our approach comprises three components: (1) annotation-guided fence placement; (2) tunable, context-aware fence-insertion heuristics; and (3) a built-in compatibility profile for per-application policy selection.

7.1 Annotation-Guided Fence Placement

As MSVC supports “volatile” metadata when generating PE binaries, we can leverage this information to guide fence placement. As most games are built for Windows, when they are emulated on Box64, this feature is particularly useful for our use case. We believe there will be a growing trend of compiler-provided annotations and real-world application support for synchronization in the future, as it is a practical way to balance performance and correctness. Our system extracts this metadata from the PE binary and places fences

around accesses to annotated memory locations. This approach is effective for ensuring correctness in critical sections of code that are explicitly marked by compilers, while avoiding the overhead of blind fence insertion across all memory operations.

7.2 Context-Aware Fence Insertion Heuristics

For applications without volatile annotations, we provide a set of heuristics for inserting memory fences around x86 memory operations during translation. These heuristics are designed to be efficient for matching real-world application needs. Similar to Microsoft’s Prism, which uses the fewest fences for TSO emulation by default [26]. We observed that most applications do not require full TSO semantics for all memory operations, especially in SIMD operations, which are often used for data-parallel computations that do not involve synchronization. Therefore, our heuristics focus on inserting fences around scalar memory operations, which are more likely to be engaged in synchronization patterns.

Figure 5 illustrates an example of TSO emulation without data races. In this example, we have two threads: a writer thread that updates two data variables (`d1` and `d2`) and a reader thread that reads these variables. The writer thread uses a lock to ensure mutual exclusion during the update, while the reader thread also uses the same lock to ensure it reads consistent values. To reduce the overhead of fence insertion, we designed 5-level TSO emulation policies (see Table 5) that can be selected based on the application’s needs. In our Pragmatic TSO emulation, we only insert fences for basic blocks containing store operations. We will insert fence `r`, `rw` before the first store operation of such basic block (entry fence), fence `w`, `w` before the last store operation of such basic block (pre-commit fence), and fence `w`, `w` after the last store operation of such basic block (post-commit fence). In this example, the fences ensure that the writes to `d1` and `d2` are visible to other threads before the lock is released, and that the lock acquisition in the reader thread happens after the writes are visible. This prevents the reader thread from observing stale values of `d1` and `d2`. This heuristic may not cover all possible synchronization patterns, but it is effective for many common cases and avoids the overhead of inserting fences for every memory operation. For some applications that require stronger TSO guarantees, we also provide a Conservative TSO mode that inserts additional fences to more closely replicate verbatim TSO semantics by inserting fence `w`, `w` after every 3 store operations in a basic block. If it still does not suffice, we also provide a Verbatim TSO mode that mimics QEMU TCG’s fence insertion strategy to ensure maximum compatibility at the cost of performance.

7.3 Built-in Compatibility Profile

To balance performance and compatibility, our system provides a built-in compatibility profile that allows users to select the appropriate TSO emulation level based on their

Initially, lock = 0, d1 = 0, d2 = 0
lock, d1, and d2 are at different addresses

<pre> ; Thread 1: Writer 1: mov al, 0 mov bl, 1 lock cmpxchg [lock], bl jne 1b ; fence (entry) mov byte ptr [d1], 1 mov byte ptr [d2], 2 ; fence (pre-commit) mov byte ptr [lock], 0 ; fence (post-commit) ret </pre>	<pre> ; Thread 2: Reader 1: mov al, 0 mov bl, 1 lock cmpxchg [lock], bl jne 1b mov dil, [d1] mov sil, [d2] ; fence (entry) mov byte ptr [lock], 0 ; fence (post-commit) ret </pre>
--	---

- fences are added during translation;
x86-TSO already forbids such reordering
- RMWs are translated with aq+rl ordering

Possible outcomes:
(dil, sil) = (0, 0) or (1, 2)

Figure 5. Example of TSO Emulation without Data Race

Table 5. TSO Emulation Levels in Our System.

Level	Name	Description
0	None (def.)	Minimum fences.
1	Pragmatic	Heuristic fences (scalar).
2	Prag. + SIMD	Level 1 + SIMD.
3	Conservative	More fences; near verbatim.
4	Verbatim	QEMU TCG-like fences.

application’s needs. Our compatibility profile will index each executable (e.g. .exe or ELF binary) or libraries (e.g. .dll or .so) to set the appropriate TSO emulation level for common applications out of the box. By now, Box64 consists of 201 entries for common applications. This profile can be easily extended by users to include additional applications as needed.

In addition to per-executable settings, our system inspects an executable’s dependent libraries and elevates the TSO emulation level to the most conservative required by any dependency, ensuring correctness even when the main executable is not listed in our compatibility profile. This is important for applications that rely on third-party libraries with stricter synchronization requirements. For example, if UnityPlayer.dll or libjvm.so is detected, the system automatically selects the Pragmatic TSO level for the entire process, as these libraries make extensive use of multithreading. This policy preserves performance for well-behaved applications while safeguarding correctness where stronger guarantees are needed.

We acknowledge the inherent limitations of our best-effort approach to TSO emulation, which does not strictly adhere

to a rigorous TSO model and thus lacks theoretical guarantees of correctness for any application. This necessitates manual effort to profile and empirically validate—through extensive testing—execution correctness and suitable TSO emulation levels, particularly for novel applications. Consequently, out-of-the-box compatibility with unknown applications remains an area for future experiments. However, as evaluated in Section 8.2, our tailored TSO emulation framework yields significant performance gains. Furthermore, it has demonstrated strong empirical compatibility (i.e., no failures observed during testing) with a wide array of complex, real-world applications (evaluated in Section 8.4).

8 Evaluation

Table 6. RISC-V Devices Used in Evaluation

Model	ISA	Cores
SOPHGO SG2044	rv64gc_vb_zv1128	64
Xiangshan Nanhu	rv64gcb	1

We conduct a comprehensive evaluation of our DBT system on two recent RISC-V platforms featuring diverse microarchitectures, as detailed in Table 6. For graphics-intensive workloads such as video games, select platforms with PCIe support were equipped with an AMD graphics card running open-source drivers.

We select representative benchmarks for single- and multithreaded performance evaluation. In addition, we tested several video games with different characteristics to demonstrate the compatibility and robustness of our system. For all open-source benchmarks, we used GCC 14.3 for all compilations. Guest x86 binaries were compiled for the x86-64-v2 microarchitecture level, which our system fully supports. The corresponding native RISC-V reference binaries were compiled from the same source code, with ISA extensions enabled per platform according to Table 6.

8.1 Single-Threaded Performance

To contextualize our system’s performance, we compare it against two primary baselines: (1) **Native RISC-V** executables, which represent the ideal performance ceiling, and (2) **QEMU user-mode emulator** (v10.0), the state-of-the-art general-purpose binary translator.

We employ the NBench benchmark suite [23] to assess the core single-threaded performance of our DBT engine. NBench is well-suited for this purpose as it contains a collection of targeted kernels designed to isolate and measure the performance of fundamental integer, memory, and FP operations. This allows for a fine-grained analysis of our JIT compiler’s code generation efficiency across different instruction classes. Its self-contained nature and reasonable execution time also facilitate robust comparisons against

slower baselines like QEMU. Performance scores are calculated by normalizing the throughput of the computation kernels against an AMD K6 233 MHz reference system, where higher scores indicate better performance. For this evaluation, all NBench variants were compiled for both x86 and RISC-V targets using GCC with the -O3 optimization level.

Table 7. DBT Optimization Level Presets

Level	Optimizations
Low	Lazy FLAGS evaluation
CallRet	Low + CALL/RET pairing
Default	Low + Macro-op fusion
High	CallRet + Default

To provide a nuanced view of performance, we evaluate our system using several optimization presets, as detailed in Table 7. These presets correspond to the techniques described in previous sections. For opportunistic acceleration, our system is configured to leverage all ISA extensions available on the host platform, ensuring a fair comparison against native RISC-V executables compiled with the same features.

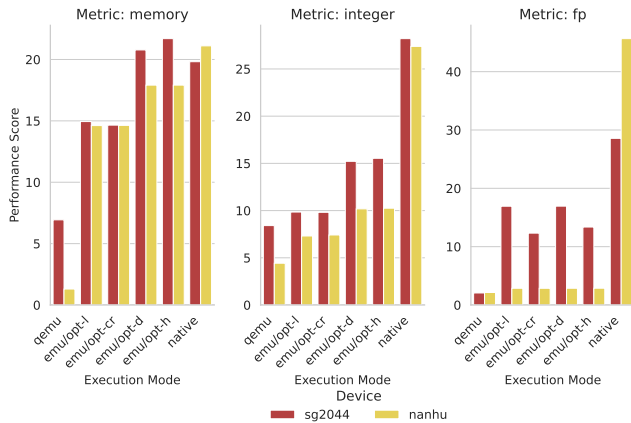


Figure 6. NBench Test Results (*emu* is our system; optimization is described in Table 7)

The single-threaded performance results on NBench are presented in Figure 6. On the SG2044 platform, which features vector extensions comparable to x86’s SSE, our system achieves an overall geometric mean performance of **67%** relative to native execution. This represents a **3.3× speedup** over QEMU’s TCG backend, demonstrating the efficiency of our JIT engine.

A breakdown of the results reveals the performance characteristics for different workload types.

- **Memory Performance:** Our system introduces minimal overhead for memory-intensive workloads, achieving near-native performance and outperforming QEMU

by 3×. This is attributable to the direct mapping of memory operations, which require little translation logic.

- **Integer Performance:** For integer-bound workloads, the emulation overhead is more pronounced due to the complexities of emulating x86’s instruction semantics and FLAGS register. Nonetheless, our JIT’s optimizations still deliver a 2× speedup over QEMU.

FP performance highlights the critical impact of host ISA feature support and the nuanced effects of our JIT optimizations. On the SG2044 platform, which features vector extensions, the translation overhead for FP workloads is comparable to that of integer workloads. In stark contrast, on the Nanhu platform, which lacks vector extensions, performance degrades significantly. The guest x86 binary, compiled with auto-vectorization, utilizes SSE instructions that our system must then de-vectorize and emulate using slow, scalar FP operations. This ISA mismatch results in a 16× performance gap compared to native execution and only a marginal improvement over QEMU.

This performance disparity underscores a key design philosophy of our system. The ability to execute this vectorized code on the Nanhu platform—albeit slowly—demonstrates the robustness of our baseline compatibility layer, which makes vector extensions optional at a predictable performance cost. Furthermore, this analysis reveals that certain DBT optimizations, while beneficial for integer-heavy code, can have a neutral or even negative impact on FP workloads. Optimizations targeting control flow, for instance, are less relevant for the data-parallel, straight-line code common in FP-intensive tasks. Recognizing this, our system provides the flexibility for users to tune optimization levels via a configuration file, allowing them to tailor the JIT’s behavior to specific workloads and achieve the best possible performance.

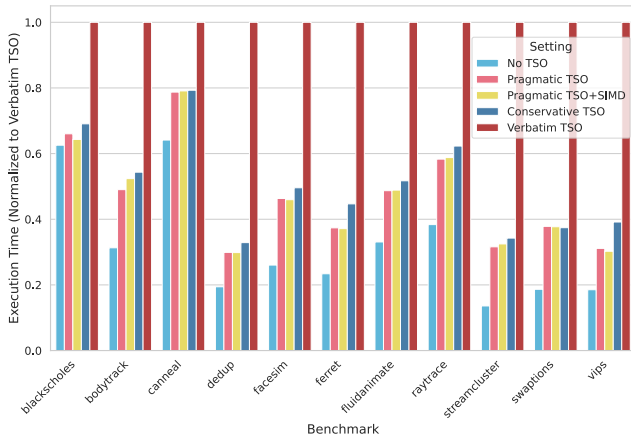
8.2 Multithreaded Performance & TSO Overhead

To quantify the performance overhead of our TSO emulation strategies, we use the PARSEC 3.0 benchmark suite [7], a standard for evaluating multiprocessor systems. Performance is measured by benchmark execution time, where lower values indicate better performance. All benchmarks were run using the *simlarge* input set with 8 threads on SG2044. The x264 benchmark was omitted due to a guest library dependency issue.

We evaluated the system under several global TSO emulation levels, detailed in Table 5. To create a fair baseline, we implemented QEMU’s conservative fence placement strategy within our own JIT (the Verbatim TSO level). This allows for a direct comparison of emulation strategies without confounding variables from using two different DBT systems. It is important to note that these levels are applied globally for this experiment, but our system also supports applying

Table 8. Compatibility and Performance of Selected x86 Desktop Games under Box64 on SG2044

Category	Game Title	Platform	Performance (FPS)		
			1% Low	Average	97% High
3D	The Witcher 3: Wild Hunt	Windows (64-bit)	7	18	23
	FlatOut 2	Windows (32-bit)	44	60	67
	Besiege	Linux (64-bit)	1	14	29
	OpenArena	Linux (64-bit)	-	47	-
2D	Hades	Windows (64-bit)	11	52	77
	Embodiment of Scarlet Devil	Windows (32-bit)	27	59	63
	Celeste	Linux (64-bit)	33	59	60
	LIMBO	Linux (32-bit)	44	57	60

**Figure 7.** PARSEC Test Results with Different TSO Emulation Levels on SG2044

them to specific memory regions for fine-grained, real-world tuning.

The performance results are presented in Figure 7. Our primary pragmatic emulation strategy (Level 1) achieves a geometric mean performance improvement of **50%** over the Verbatim TSO (Level 4) baseline, much better than 7.3% reported in the previous literature [15]. This demonstrates the effectiveness of our heuristic-based approach. However, Level 1 still incurs a significant performance overhead when compared to the default None (Level 0) configuration. This result underscores the critical need for a more fine-grained approach, motivating our system’s support for *regional* TSO emulation, where these protections can be applied selectively only where needed, rather than globally.

8.3 Overhead Breakdown

A series of targeted micro-experiments was designed to evaluate the overhead associated with code generation and dynamic library acceleration.

8.3.1 Code Generation Overhead. Box64’s lightweight code generation framework introduces reasonable performance overhead. We assessed this overhead using Linux perf. In the NBench benchmark, the JIT engine accounted for less than 0.01% of the execution time, making it essentially negligible. However, in typical workloads of the JIT virtual machine, the proportion of time dedicated to code generation increases considerably. For example, when running SciMark 2.0a [29] with LuaJIT 2.1 [30], code generation consumed 1% of total time.

8.3.2 Dynamic Library Acceleration. By forwarding function calls from commonly used dynamic libraries to their native implementations, Box64 not only eliminates the dependency on a complete x86 root filesystem but also reduces the portion of the program requiring emulation, thereby enhancing overall performance. For instance, in bzip2 [32] compression tests, utilizing the native libbz2.so.1 yielded a performance improvement of 1.38× compared to the emulated version. However, library forwarding does not invariably produce positive outcomes. In the NBench workload, for example, employing the native libm.so.6 resulted in floating-point scores that were only 0.8× of those obtained through emulation. This degradation can be attributed to both the implementation quality of the RISC-V math library and the overhead introduced by the function forwarding layer.

8.4 Application Compatibility and Robustness

To evaluate real-world application compatibility and robustness, we use a curated suite of desktop video games as our primary benchmark. This workload is ideal for several reasons: (1) most commercial titles⁶ are closed-source and exclusively available as x86 binaries; (2) their complex, multithreaded nature and reliance on GPU driver interoperability make

⁶The authors share a passion for games, which serve as ideal examples to demonstrate both the performance and compatibility advantages of our framework. All games were purchased from commercial websites, such as GOG, or physical stores. Testing was conducted solely for research purposes in the lab, not for any commercial purposes.

them a strenuous stress test; and (3) they represent a class of applications that general-purpose emulators like QEMU user-mode cannot typically run. Our diverse test suite, detailed in Table 8, includes 2D and 3D titles, 32-bit and 64-bit binaries, and games designed for both Windows and Linux.

All tests were conducted on the SG2044 platform with default Box64 settings. To create a demanding and representative workload, all games were configured to a 1080p resolution⁷ with the highest available graphics quality settings. Windows titles were run using the x86 versions of Wine 11.1⁸ [4] and DXVK 2.7.1 [11], demonstrating crucial interoperability with these compatibility layers. Performance was captured over a one-hour gameplay session using MangoHud [13], reporting the average, 1% low, and 97% high frames per second (FPS) values. FPS measures the frequency at which consecutive images are displayed; it serves as a primary indicator of visual smoothness and system responsiveness, where higher is better. Vertical synchronization had limited the maximum performance to around 60 FPS.

Our system demonstrates the capability to run demanding 3D titles. The modern AAA game *The Witcher 3: Wild Hunt* runs at a playable, albeit capped, framerate of approximately 15–20 FPS. CPU-intensive simulation games like *Besiege* are heavily CPU-bound, resulting in lower framerates; we observed an average of 14 FPS in a small scene, and performance would be even poorer in bigger ones. For older, less demanding 3D titles such as *FlatOut 2*, our system achieves a stable 60 FPS. Critically, for the open-source benchmark *OpenArena*, the emulated x86 version runs at 67% of the performance of a native RISC-V build, providing a direct measure of our translation overhead on a complete application.

Performance on less graphically intensive 2D titles is significantly stronger. Many games, including the classic *Embodiment of Scarlet Devil* and the fast-paced *Celeste*, consistently reach the 60 FPS performance cap. Notably, *Celeste* employs the Mono .NET runtime with frequent JIT compilation activity, thereby validating the efficiency and correctness of our system's SMC handling. Nonetheless, even some 2D titles like *Hades* can remain CPU-bound, highlighting that performance is a function of game engine design, not just graphical complexity. Overall, our system successfully runs a diverse range of unmodified, closed-source x86 games, demonstrating a level of compatibility and robustness currently unmet by previous DBT solutions for RISC-V.

9 Conclusion

In this paper, we presented the RISC-V implementation of Box64, featuring a dynamic binary translator with high performance and compatibility that bridges the critical software

gap between the mature x86-64 ecosystem and emerging RISC-V platforms. Our work makes three primary contributions: (1) a JIT engine incorporating a suite of advanced optimizations for high single-threaded performance; (2) a unified framework for bridging the significant architectural gaps between the ISAs, ensuring robustness across the fragmented RISC-V hardware landscape; and (3) a novel, pragmatic TSO emulation method that ensures multithreaded correctness for real-world applications with minimal overhead. Our evaluation demonstrates that Box64 makes running unmodified, complex x86 desktop software on RISC-V a practical reality, and we believe our TSO emulation principles can inform future translators for other weakly-ordered architectures.

Acknowledgments

We would like to thank the anonymous reviewers, and our shepherd, Diogo Behrens, for their insightful comments and constructive feedback. This work was supported in part by the National Natural Science Foundation of China (under Grant #T2125006) and Guangdong S&T Program (under Grant #2025B0101080001). We are grateful to Han Gao for his assistance with computational resources. We also thank the Institute of Software, Chinese Academy of Sciences for financial support. Finally, we would like to acknowledge the RISC-V community for maintaining the invaluable open-source ecosystem.

References

- [1] Apple Inc. 2020. About the Rosetta Translation Environment. <https://developer.apple.com/documentation/apple-silicon/about-the-rosetta-translation-environment>. Accessed: 2024-11-26.
- [2] Apple Inc. 2020. Accelerating the performance of Rosetta. <https://developer.apple.com/documentation/virtualization/accelerating-the-performance-of-rosetta>. Accessed: 2025-05-09.
- [3] Arm. 2016. Load-Acquire and Store-Release instructions. <https://developer.arm.com/documentation/102336/0100/Load-Acquire-and-Store-Release-instructions>. Accessed: 2025-05-11.
- [4] Wine authors. 1993. WineHQ - Run Windows applications on Linux, BSD, Solaris and macOS. <https://www.winehq.org/>. Accessed: 2024-11-30.
- [5] Martin Beck, Koustubha Bhat, Lazar Stričević, Geng Chen, Diogo Behrens, Ming Fu, Viktor Vafeiadis, Haibo Chen, and Hermann Härtig. 2023. AtoMig: Automatically Migrating Millions Lines of Code from TSO to WMM. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, Vancouver BC Canada, 61–73. doi:10.1145/3575693.3579849
- [6] Fabrice Bellard. 2005. QEMU, a fast and portable dynamic translator. In *USENIX annual technical conference, FREENIX Track*, Vol. 41. California, USA, 10–5555.
- [7] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. 2008. The PARSEC benchmark suite: characterization and architectural implications. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques* (Toronto, Ontario, Canada) (PACT '08). Association for Computing Machinery, New York, NY, USA, 72–81. doi:10.1145/1454115.1454128
- [8] Derek Bruening and Saman Amarasinghe. 2004. *Efficient, transparent, and comprehensive runtime code manipulation*. Ph. D. Dissertation.

⁷Excluding *Embodiment of Scarlet Devil*.

⁸The evaluation employs an Ntsync-enabled Wine with kernel support, which reduces Inter-process Communication (IPC) overhead.

- Massachusetts Institute of Technology.
- [9] Vitaly Chipounov, Volodymyr Kuznetsov, and George Candea. 2012. The S2E platform: Design, implementation, and applications. *ACM Transactions on Computer Systems (TOCS)* 30, 1 (2012), 1–49.
 - [10] Amanieu d'Antras, Cosmin Gorgovan, Jim Garside, and Mikel Luján. 2016. Optimizing indirect branches in dynamic binary translators. *ACM Transactions on Architecture and Code Optimization (TACO)* 13, 1 (2016), 1–25.
 - [11] doitsujin. 2017. DXVK. <https://github.com/doitsujin/dxvk>. Accessed: 2025-05-11.
 - [12] FEX-Emu. 2021. FEX: A fast usermode x86 and x86-64 emulator for Arm64 Linux. <https://github.com/FEX-Emu/FEX>. Accessed: 2024-11-28.
 - [13] flightlessmango. 2025. MangoHud: A Vulkan and OpenGL overlay for monitoring FPS, temperatures, CPU/GPU load and more. <https://github.com/flightlessmango/MangoHud>. Accessed: 2025-09-24, Version 0.8.1.
 - [14] Sheng-Yu Fu, Jan-Jan Wu, and Wei-Chung Hsu. 2015. Improving SIMD code generation in QEMU. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1233–1236.
 - [15] Chen Gao, Xiangwei Meng, Wei Li, Jinhui Lai, Yiran Zhang, and Fengyuan Ren. 2024. CrossMapping: Harmonizing Memory Consistency in Cross-ISA Binary Translation. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 1013–1028.
 - [16] Redha Gouicem, Dennis Sprockholt, Jasper Ruehl, Rodrigo CO Rocha, Tom Spink, Soham Chakraborty, and Pramod Bhatotia. 2022. Risotto: a dynamic binary translator for weak memory model architectures. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 107–122.
 - [17] Yu-Chuan Guo, Wu Yang, Jiunn-Yeu Chen, and Jenq-Kuen Lee. 2016. Translating the ARM Neon and VFP instructions in a binary translator. *Software: Practice and Experience* 46, 12 (2016), 1591–1615.
 - [18] Ding-Yong Hong, Chun-Chen Hsu, Pen-Chung Yew, Jan-Jan Wu, Wei-Chung Hsu, Pangfeng Liu, Chien-Min Wang, and Yeh-Ching Chung. 2012. HQEMU: a multi-threaded and retargetable dynamic binary translator on multicores. In *Proceedings of the Tenth International Symposium on Code Generation and Optimization*. 104–113.
 - [19] Huawei. 2023. Huawei Dynamic Binary Translator (ExaGear). <https://www.hikunpeng.com/en/developer/devkit/download/exagear>. Accessed: 2024-12-06.
 - [20] Wei Li, Xiaohui Luo, Yiran Zhang, Qingkai Meng, and Fengyuan Ren. 2022. CrossDBT: an LLVM-based user-level dynamic binary translation emulator. In *European Conference on Parallel Processing*. Springer, 3–18.
 - [21] Xinyu Li, Guangyao Guo, Yanzi Lan, Feng Xue, Chenji Han, Gen Niu, and Fuxin Zhang. 2024. Tiaozhuan: A General and Efficient Indirect Branch Optimization for Binary Translation. *ACM Trans. Archit. Code Optim.* (Nov. 2024). doi:10.1145/3703355 Just Accepted.
 - [22] Haoming Lin, Yong Dong, Wanqing Chi, Zhenwei Wu, and Hongqing Zeng. 2023. Efficient Dynamic Binary Translation with Accumulative Persistent Code Caching. In *2022 IEEE 28th International Conference on Parallel and Distributed Systems (ICPADS)* (2023-01). 458–466. doi:10.1109/ICPADS56603.2022.00066 ISSN: 2690-5965.
 - [23] Uwe F Mayer. 2017. Linux/Unix nbench. <https://www.math.utah.edu/~mayer/linux/bmark.html>. Accessed: 2025-09-24.
 - [24] Paul E McKenney. 2010. Memory Barriers: a Hardware View for Software Hackers. <http://www.rdrop.com/users/paulmck/scalability/paper/whymb.2010.06.07c.pdf>. Accessed: 2024-12-02.
 - [25] Microsoft. 2024. How emulation works on Arm. <https://learn.microsoft.com/en-us/windows/arm/apps-on-arm-x86-emulation>. Accessed: 2024-11-26.
 - [26] Microsoft. 2024. Program Compatibility Troubleshooter on Arm. <https://learn.microsoft.com/en-us/windows/arm/apps-on-arm-program-compat-troubleshooter>.
 - [27] Microsoft. 2024. /volatileMetadata (Generate metadata on volatile memory accesses). <https://learn.microsoft.com/en-us/cpp/build/reference/volatile?view=msvc-170>. Accessed: 2025-05-11.
 - [28] LAT opensource. 2021. Loongson Architecture Translator for x86. <https://github.com/lat-opensource/lat>. Accessed: 2025-05-09.
 - [29] Mike Pall. 2010. Lua SciMark. <https://luajit.org/download/scimark.lua>. Accessed: 2026-02-10.
 - [30] Mike Pall. 2026. The LuaJIT Project. <https://luajit.org>. Accessed: 2026-02-10.
 - [31] ptitSeb. 2021. Box64 - Linux Userspace x86_64 Emulator with a twist, targeted at ARM64 Linux devices. <https://github.com/ptitSeb/box64>. Accessed: 2024-11-28.
 - [32] Julian Seward. 1996–2019. bzip2 and libbzip2. <https://sourceware.org/bzip2/>. Accessed: 2026-02-11.
 - [33] Tom Spink and Björn Franke. 2024. Accelerating Shared Library Execution in a DBT. In *Proceedings of the 25th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*. 1–11.
 - [34] Jie Tan, Jian-min Pang, and Shuai-bing Lu. 2017. Using Local Library Function in Binary Translation. In *Current Trends in Computer Science and Mechanical Automation Vol.1*, Shawn X. Wang (Ed.). De Gruyter Open, 123–132. doi:10.1515/9783110584974-016
 - [35] Andrew Waterman, Yunsup Lee, David A. Patterson, and Krste Asanović. 2014. The RISC-V Instruction Set Manual, Volume I: User-Level ISA. <https://github.com/riscv/riscv-isa-manual> Accessed: 2024-11-28.
 - [36] Matthias Wenzl, Georg Merzdovnik, Johanna Ullrich, and Edgar Weippl. 2019. From hack to elaborate technique—a survey on binary rewriting. *ACM Computing Surveys (CSUR)* 52, 3 (2019), 1–37.
 - [37] XUANTIE-RV. 2022. XuanTie vendor extension Instruction Set spec. <https://github.com/XUANTIE-RV/thread-extension-spec>. Accessed: 2024-11-30.
 - [38] Niko Zurschraen, Nils Bosbach, Jan Moritz Joseph, Lukas Jünger, Jan Henrik Weinstock, and Rainer Leupers. 2023. Efficient RISC-V-on-x64 Floating Point Simulation. In *2023 IEEE 41st International Conference on Computer Design (ICCD)*. IEEE, 558–565.

A Artifact Appendix

A.1 Abstract

This artifact comprises the RISC-V backend implementation for Box64, accompanied by benchmarking suites and application demonstrations. Specifically, it includes:

- The Box64 source code with the optimized RISC-V backend.
- **NBench**: Precompiled x86-64 and RISC-V binaries, execution scripts, logs, and validation screencasts.
- **PARSEC**: Reproducible execution scripts, performance logs, and validation screencasts (benchmark files omitted due to size).
- **Applications**: Screencasts of real-world gaming workloads featuring real-time FPS overlays.

As the SG2044 and Nanhu platforms are not generally available and we are unable to offer remote access, we provide comprehensive screencasts and raw execution logs to verify our reported results. Furthermore, we provide portable scripts to facilitate the reproduction of NBench and PARSEC experiments on any compatible RISC-V device.

A.2 Description & Requirements

A.2.1 How to Access. Our work is publicly upstreamed to the original Box64 project, available at:

<https://github.com/ptitSeb/box64>

The artifact corresponding to this paper is archived at

<https://doi.org/10.5281/zenodo.18611830>

A.2.2 Software Dependencies. A standard 64-bit Linux distribution is required. For Windows application emulation, we recommend **Linux 6.14+** and **Wine 11+** to leverage **NTsync** support for improved synchronization performance. Graphics workloads require **Mesa** drivers and **DXVK**. Performance monitoring is conducted via **MangoHud**.

A.2.3 Hardware Dependencies. The artifact requires 64-bit RISC-V hardware with at least the **RV64G** instruction set. Performance scales with the availability of additional hardware extensions. To evaluate gaming workloads, the RISC-V platform must feature a PCIe interface equipped with a discrete AMD GPU supporting OpenGL and Vulkan (e.g., AMD Radeon RX 7900 XTX).

A.3 Evaluation Workflow

A.3.1 Major Claims.

- Box64 demonstrates a significant emulation speed advantage over QEMU-TCG.
- Box64 reduces TSO emulation overhead while maintaining high execution fidelity.
- Box64 is capable of executing complex, real-world x86-64 applications on RISC-V platforms.

A.3.2 Experiments.

Preparation. Building Box64, NBench, and PARSEC from source requires approximately 5 human-minutes of configuration and 60 compute-minutes. Downloading the games (~70GB) is dependent on network bandwidth.

Benchmarks. Executing the NBench and PARSEC suites requires approximately 2 human-minutes of setup and 60 compute-minutes.

Applications. Manual validation of gaming workloads requires approximately 90 human-minutes. While some applications offer automated replay modes (~10 compute-minutes), most demonstrations require manual interaction to advance through the game states.